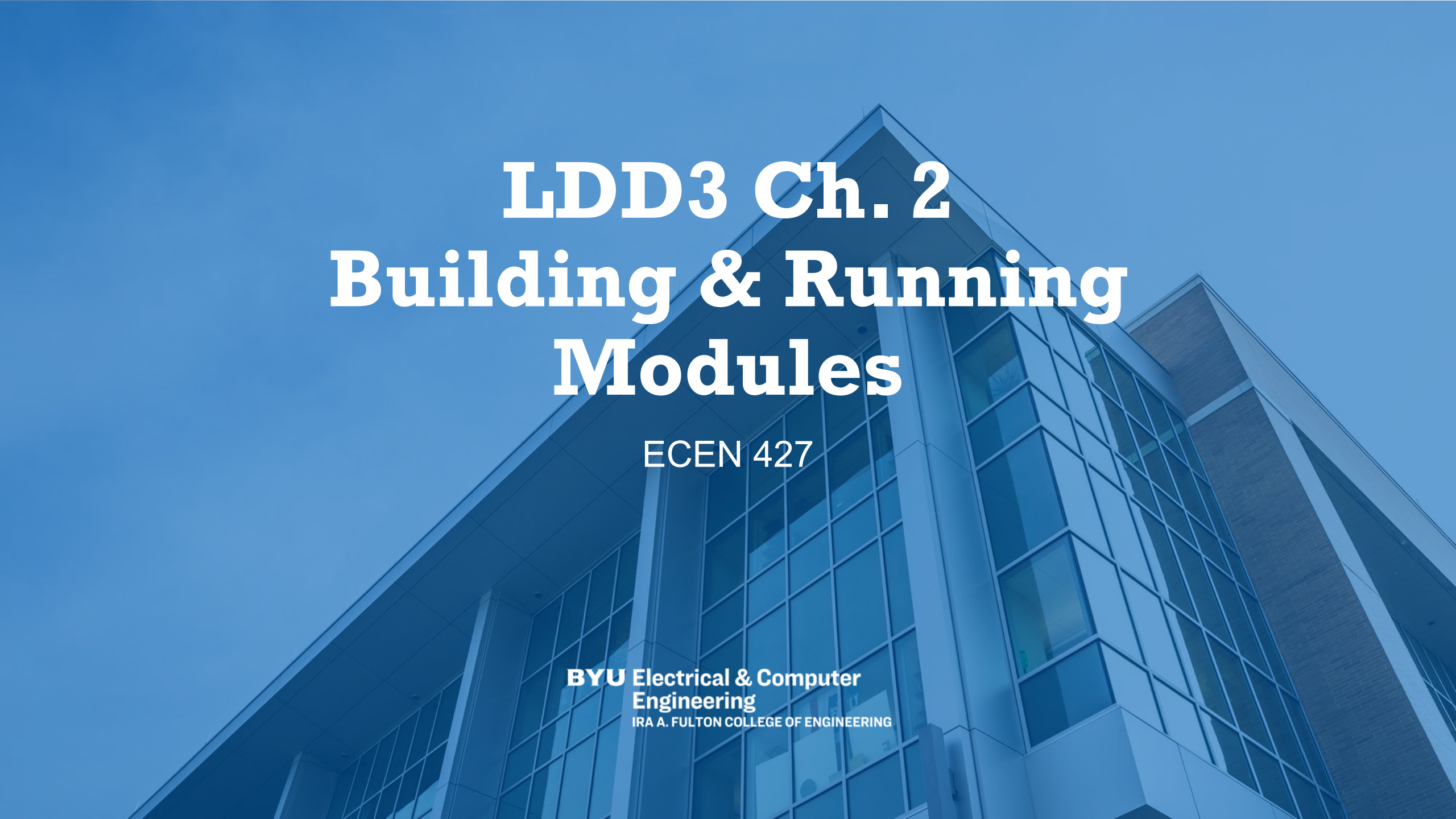




LDD3 Ch. 2

Building & Running

Modules

ECEN 427

BYU Electrical & Computer
Engineering
IRA A. FULTON COLLEGE OF ENGINEERING

Learning Objectives

- Write, build, load, and unload a minimal kernel module
- Describe how a module differs from a program: no main, no C library, no large stack, no floating point
- Explain why kernel code must be reentrant and tell process context from interrupt context
- Write an init function that undoes its work when something fails partway through

The Hello World Module

```
#include <linux/init.h>
#include <linux/module.h>
MODULE_LICENSE("Dual BSD/GPL");

static int hello_init(void)
{
    printk(KERN_ALERT "Hello, world\n");
    return 0;
}

static void hello_exit(void)
{
    printk(KERN_ALERT "Goodbye, cruel world\n");
}

module_init(hello_init);
module_exit(hello_exit);
```

The Hello World Module

```
#include <linux/init.h>
#include <linux/module.h>
MODULE_LICENSE("Dual BSD/GPL");
```

Different headers than user space

GPL License

```
static int hello_init(void)
{
    printk(KERN_ALERT "Hello, world\n");
    return 0;
}
```

printk instead of printf

```
static void hello_exit(void)
{
    printk(KERN_ALERT "Goodbye, cruel world\n");
}
```

Macros for init/exit

```
module_init(hello_init);
module_exit(hello_exit);
```

There is no main!

Kernel Modules

The Hello World Module

```
#include <linux/init.h>
#include <linux/module.h>
MODULE_LICENSE("Dual BSD/GPL");

static int hello_init(void)
{
    printk(KERN_ALERT "Hello, world\n");
    return 0;
}

static void hello_exit(void)
{
    printk(KERN_ALERT "Goodbye, cruel world\n");
}

module_init(hello_init);
module_exit(hello_exit);
```

Different headers than user space

GPL License

printk instead of printf

Macros for init/exit

There is no main!

Where is main()?

While most small and medium-sized applications perform a single task from beginning to end, every kernel module just registers itself in order to serve future requests, and its initialization function terminates immediately...

In other words, the task of the module's initialization function is to prepare for later invocation of the module's functions; it's as though the module were saying, "Here I am, and this is what I can do."

The module's exit function (`hello_exit` in the example) gets invoked just before the module is unloaded. It should tell the kernel, "I'm not there anymore; don't ask me to do anything else."

This kind of approach to programming is similar to **event-driven programming**, but while not all applications are event-driven, each and every kernel module is.

Where is the kernel source code

- Official repo: <https://github.com/torvalds/linux>
- Easily searchable here: <https://elixir.bootlin.com/linux/latest/source>

What version are we running?

- `uname -r`

Kernel source for our version:

<https://elixir.bootlin.com/linux/v6.6.10/source>

Linking

The Hello World Module

```
#include <linux/init.h>
#include <linux/module.h>
MODULE_LICENSE("Dual BSD/GPL");

static int hello_init(void)
{
    printk(KERN_ALERT "Hello, world\n");
    return 0;
}

static void hello_exit(void)
{
    printk(KERN_ALERT "Goodbye, cruel world\n");
}

module_init(hello_init);
module_exit(hello_exit);
```

Different headers than user space

GPL License

printk instead of printf

Macros for init/exit

There is no main!

As a programmer, you know that an application can call functions it doesn't define: the linking stage resolves external references using the appropriate library of functions.

- **printf** is one of those callable functions and is defined in **libc**.
- A module, on the other hand, is linked only to the kernel, and the only functions it can call are the ones exported by the kernel; there are no libraries to link to.
- The **printk** function used in **hello.c** earlier, for example, is the version of **printf** defined within the kernel and exported to modules. It behaves similarly to the original function, with a few minor differences, the main one being lack of floating-point support.
- Because no library is linked to modules, source files should never include the usual header files, `<stdarg.h>` and very special situations being the only exceptions.

```
1 TARGET_MODULE=audio_codec
2
3 ccflags-y := -std=gnu99 -Wno-declaration-after-statement
4
5 # If we are running by kernel building system
6 ifneq ($(KERNELRELEASE),)
7     obj-m := $(TARGET_MODULE).o
8 # If we running without kernel build system
9 else
10     BUILDSYSTEM_DIR:=/lib/modules/$(shell uname -r)/build
11     PWD:=$(shell pwd)
12
13 all :
14     # run kernel build system to make module
15     $(MAKE) -C $(BUILDSYSTEM_DIR) M=$(PWD) modules
16
17 clean:
18     # run kernel build system to cleanup in current directory
19     $(MAKE) -C $(BUILDSYSTEM_DIR) M=$(PWD) clean
20
21 install:
22     $(MAKE) -C $(BUILDSYSTEM_DIR) M=$(PWD) modules_install
23     depmod -A
24
25 endif
```

Inserting & Removing Modules

```
dmesg // Display kernel logs
sudo dmesg --clear // Clear kernel logs

cd kernel/helloworld
make // Build module
sudo insmod helloworld.ko // Insert module
dmesg
lsmod // List inserted modules

sudo rmmod helloworld // Remove module
dmesg
```

- A .ko is linked against one exact kernel build
- The compiler bakes a vermagic string into the module: kernel release, SMP, preempt, architecture, and a few config options
- At load time the kernel compares that string to its own; any mismatch and the module is refused
- This is why the Makefile builds against **/lib/modules/\$(uname -r)/build**: the headers and config of the kernel that is actually running
- Rebuild your module after any kernel update; the source did not change, but the kernel it must fit did

View the vermagic:

```
modinfo ./helloworld.ko
```

Kernel Module Considerations

Program/Code Segments

Segment	Bare Metal	Linux User Space	Kernel Module
1.			
2.			
3.			
4.			

Program/Code Segments

Segment	Bare Metal	Linux User Space	Kernel Module
1. Program Code	Size fixed and allocated at link time.	Size fixed at link time, memory allocated at load time.	Size fixed at link time, memory allocated at load time.
2. Global data	Size fixed and allocated at link time.	Size fixed at link time, memory allocated at load time.	Size fixed at link time, memory allocated at load time.
3. Stack	Compile-time allocated space for stack and heap. They grow toward each other.	Grows on demand at run time. Limit set by the OS, overflow is a segfault.	No stack of its own. Borrows the caller's fixed 8 KB kernel stack. Overflow may corrupt memory silently.
4. Heap		Grows on demand via malloc / brk / mmap. Nothing to collide with, malloc returns NULL when it fails.	No malloc. Use kmalloc / kfree (or vmalloc for large buffers). Runtime allocated from the kernel's allocators.

Applications: are laid out in virtual memory with a very large stack area.

The stack, of course, is used to hold the function call history and all automatic variables created by currently active functions.

The kernel, instead, has a very small stack; it can be as small as a single, 4K (or 8K/16K) page.

Your functions must share that stack with the entire kernel-space call chain.

Thus, it is never a good idea to declare large automatic variables; if you need larger structures, you should allocate them dynamically at call time.

Kernel code cannot do floating point arithmetic.

Enabling floating point would require that the kernel save and restore the floating point processor's state on each entry to, and exit from, kernel space—at least, on some architectures.

Given that there really is no need for floating point in kernel code, the extra overhead is not worthwhile.

Concurrency in the Kernel

- Your code so far ran one thread, top to bottom. A driver does not get that luxury.
- Your driver can be entered from several directions at once:
 - Several processes calling open / read / write / ioctl on your device at the same time
 - A hardware interrupt firing in the middle of any of them
 - Timers, work queues, and other kernel threads (Ch 7)
- On the PYNQ's two cores, "at the same time" is literal. The kernel can also preempt your code at almost any point.
- There is one copy of your module, so every global and static variable is shared by all of them.
 - Kernel code must be reentrant: it must behave correctly when run by more than one context at once.
 - Rule 1: avoid shared data. Rule 2: when you cannot, lock it (Ch 5).

Which process am I in? The current pointer

- Code that runs because of a system call runs in process context: on behalf of the calling process, in its address space, on its kernel stack.
- **current** points to that process's struct task_struct (in <linux/sched.h>).

```
printk(KERN_INFO "The process is %s (pid %i)\n",  
        current->comm, current->pid);
```

(Let's add this to hello_world and try it out)

- Interrupt handlers are not in process context. current is whatever happened to be running, so do not use it there.

User space vs Kernel Drivers

The advantages of user-space drivers are:

- The full C library can be linked in.
- The programmer can run a conventional debugger on the driver code without having to go through contortions to debug a running kernel.
- If a user-space driver hangs, you can simply kill it. Problems with the driver are unlikely to hang the entire system, unless the hardware being controlled is really misbehaving.
- User memory is swappable, unlike kernel memory. An infrequently used device with a huge driver won't occupy RAM that other programs could be using, except when it is actually in use.
- A well-designed driver program can still, like kernel-space drivers, allow concurrent access to a device.
- If you must write a closed-source driver, the user-space option makes it easier for you to avoid ambiguous licensing situations and problems with changing kernel interfaces.

User space vs Kernel Drivers

But the user-space approach to device driving has a number of drawbacks. The most important are:

- Interrupts are not directly available in user space.
- Direct access to memory is possible only by mmaping /dev/mem (or via another kernel module), and only a privileged user can do that.
- Response time is slower, because a context switch is required to transfer information or actions between the client and the hardware.
- Worse yet, if the driver has been swapped to disk, response time is unacceptably long. Using the mlock system call might help, but usually you'll need to lock many memory pages, because a user-space program depends on a lot of library code. mlock, too, is limited to privileged users.
- The most important devices can't be handled in user space, including, but not limited to, network interfaces and block devices.

Exiting & Error Handling

Module Cleanup

While an application that terminates can be lazy in releasing resources or avoids clean up altogether, the **exit function of a module must carefully undo everything the init function built up**, or the pieces remain around until the system is rebooted.

Incidentally, the ability to unload a module is one of the features of modularization that you'll most appreciate, because it helps cut down development time; you can test successive versions of your new driver without going through the lengthy shutdown/reboot cycle each time.

- Applications in user space can be lazy about resources
 - Allocated memory / files are released back to the OS when the process terminates
- Kernel modules are not processes. Any resources they allocate are reserved indefinitely and not necessarily tied to the module.
- If they are unloaded without deallocating resources, those may be lost forever until reboot.

Error Handling w/ goto

“Error recovery is sometimes best handled with the goto statement. We normally hate to use goto, but in our opinion, this is one situation where it is useful.”

```
int __init my_init_function(void)
{
    int err;

    /* registration takes a pointer and a name */
    err = register_this(ptr1, "skull");
    if (err) goto fail_this;
    err = register_that(ptr2, "skull");
    if (err) goto fail_that;
    err = register_those(ptr3, "skull");
    if (err) goto fail_those;

    return 0; /* success */

fail_those: unregister_that(ptr2, "skull");
fail_that: unregister_this(ptr1, "skull");
fail_this: return err; /* propagate the error */
}
```

```
void __exit my_cleanup_function(void)
{
    unregister_those(ptr3, "skull");
    unregister_that(ptr2, "skull");
    unregister_this(ptr1, "skull");
    return;
}
```

Error Handling w/ item tracking

```
int __init my_init(void)
{
    int err = -ENOMEM;

    item1 = allocate_thing(arguments);
    item2 = allocate_thing2(arguments2);
    if (!item1 || !item2)
        goto fail;
    err = register_stuff(item1, item2);
    if (!err)

        stuff_ok = 1;
    else
        goto fail;
    return 0; /* success */

fail:
    my_cleanup();
    return err;
}
```

```
struct something *item1;
struct somethingelse *item2;
int stuff_ok;

void my_cleanup(void)
{
    if (item1)
        release_thing(item1);
    if (item2)
        release_thing2(item2);
    if (stuff_ok)
        unregister_stuff();
    return;
}
```